

Introduction to Deep Learning

CS 312 Deep Learning Alchemy · Ungraded

 Many aspects of this document are AI-generated, direct questions to course staff as they come up

How to read this handout. This handout introduces the concepts behind our language-model training setup. If a term is unfamiliar, in addition to asking course staff, you can also search online or ask a chatbot, then compare the explanation with the surrounding text and the code. You do not need to understand every detail immediately: experiments across the assignments will help build that intuition. The goal of this class is to learn how to run experiments that determine the role of each component of a model and its training recipe.

The main text explains general concepts. Purple notes at the end of each subsection connect those concepts to our settings and configuration fields.

Contents

1	Objective	3
1.1	The Prediction Problem	3
1.2	Autoregressive Modeling	3
1.3	Cross-Entropy Loss	3
1.4	Held-Out Evaluation	4
2	Data	4
2.1	Tokenization, Vocabulary, and Special Tokens	4
2.2	Concatenation, Chunking, and Context Length	5
2.3	Dataset Size, Subsets, and Storage	5
2.4	Ordering and Data Seeds	6
3	Optimization	6
3.1	Minibatches and Gradient Descent	6
3.2	Backpropagation and Gradient Accumulation	6
3.3	Epochs and the Training Budget	7
3.4	Learning Rate and Scheduling	7
3.5	SGD with Momentum	8
3.6	Adam and Adaptive Updates	8
3.7	Weight Decay and Parameter Groups	9
3.8	Gradient Clipping	9
3.9	Numerical Precision and Execution	9
3.10	Random Seeds and Determinism	10

- 3.11 Diagnostics and Experiment Records 10
 - 3.12 Initialization from Checkpoints and Resuming 11
- 4 Architecture 11
 - 4.1 Tensors, Dimensions, and Model Size 11
 - 4.2 Overall Transformer Computation 12
 - 4.3 Embedding Lookup 13
 - 4.4 Residual Connections and Pre-Normalization 13
 - 4.5 RMS Normalization 14
 - 4.6 Single-Head Causal Attention 14
 - 4.7 Multi-Head Attention 15
 - 4.8 Rotary Position Embeddings 16
 - 4.9 Query/Key Normalization 16
 - 4.10 The SwiGLU MLP 17
 - 4.11 Unembedding, Softmax, and Weight Tying 17
 - 4.12 Dropout and Training versus Evaluation 17
 - 4.13 Parameter Initialization 18
 - 4.14 Scaling Model Depth 18

1 Objective

1.1 The Prediction Problem

A language model assigns probabilities to sequences of tokens. Tokens are discrete units representing text; we discuss their construction in Section 2. Write a sequence as $x = (x_1, \dots, x_T)$, where each token ID belongs to a vocabulary of size V . The model has trainable parameters θ , such as the entries of its weight matrices. These parameters determine which sequences receive high probability.

The training data is a finite sample from a distribution p_{data} . Learning should capture patterns that also occur in new samples from that distribution, not merely assign high probability to the specific training examples. This distinction motivates evaluating on data that is not used for parameter updates.

Next-token prediction on a large text collection is called *pretraining*. With sufficient data and model capacity, pretraining can learn representations useful for other tasks. Models can subsequently be adapted through supervised fine-tuning or reinforcement learning. Lower pretraining loss measures better prediction on the evaluated distribution; it does not by itself measure every downstream capability.

Our setting. We study pretraining on internet documents from DCLM. Our primary outcome is held-out next-token prediction loss. The default recipe does not include instruction tuning, reinforcement learning, or a separate supervised task.

1.2 Autoregressive Modeling

An autoregressive model predicts each token using only preceding tokens. By the probability chain rule, a full sequence likelihood factors as $p_{\theta}(x_1) \prod_{t=1}^{T-1} p_{\theta}(x_{t+1} \mid x_{\leq t})$. Here $x_{\leq t}$ denotes the prefix $(x_1, \dots, x_t)$. If the first token is given as context, the modeled likelihood is instead

$$p_{\theta}(x_{2:T} \mid x_1) = \prod_{t=1}^{T-1} p_{\theta}(x_{t+1} \mid x_{\leq t}).$$

We abbreviate this conditional likelihood as $p_{\theta}(x)$ below. This convention avoids assigning a loss to a token for which the row supplies no preceding context.

During training, the model receives the true prefix, not tokens sampled from its own predictions. This is sometimes called *teacher forcing*. A causal transformer can compute predictions for all positions in parallel while masking information from future positions. Parallel computation does not change the autoregressive factorization. At generation time, in contrast, new tokens would be produced sequentially and appended to the prefix.

Our setting. For a row of length $T = 1024$, we compare the outputs at positions $1, \dots, 1023$ with the actual tokens at positions $2, \dots, 1024$. There are 1023 prediction targets per row. The first token is context; the final output has no target within this row. We do not train a separate first-token predictor or sample tokens during training.

1.3 Cross-Entropy Loss

Maximum-likelihood training seeks parameters that assign high probability to the observed data. Minimizing negative log likelihood is equivalent: the logarithm preserves the ordering of likelihoods and

converts a product of probabilities into a sum. Our population objective, normalized per target token, is

$$\begin{aligned} L(\theta) &= \mathbb{E}_{x \sim p_{\text{data}}} \left[-\frac{1}{T-1} \log p_{\theta}(x) \right] \\ &= \mathbb{E}_{x \sim p_{\text{data}}} \left[-\frac{1}{T-1} \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t}) \right]. \end{aligned}$$

This is *cross entropy*: for each prediction, the target places all its mass on the observed next token, so its cross entropy with the model is just minus the log probability of that token. It is not the probability itself. The loss penalizes assigning very small probability to an observed event.

For one row, denote the expression inside the expectation by $\ell(\theta; x)$. We cannot calculate the population expectation exactly; training and evaluation both approximate it with averages over available examples. With natural logarithms, loss is measured in *nats per token*. The quantity $\exp(L)$ is called perplexity. Comparisons of per-token losses require care when tokenizers or data distributions differ.

Our setting. Our loss is the mean next-token negative log probability, with natural logs, over all target positions in a batch. There is no label smoothing, padding loss mask, or auxiliary objective in the default transformer. Log probabilities and their reduction are computed in fp32. Weight decay is handled by the optimizer, not added to the reported cross-entropy loss.

1.4 Held-Out Evaluation

A validation set contains examples withheld from gradient updates. At a fixed parameter value, its average loss estimates the population objective:

$$\hat{L}_{\text{val}}(\theta) = \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} \ell(\theta; x^{(i)}).$$

This expression assumes equal-length rows; more generally we weight by the number of prediction targets. Evaluating periodically reveals how prediction quality changes as training proceeds. Evaluation does not update parameters or optimizer state, and stochastic training mechanisms such as dropout are disabled.

A training-batch loss is noisy because each step uses different examples. Validation on a fixed collection makes checkpoints easier to compare, although the estimate still has finite-sample uncertainty. Repeatedly choosing designs based on the same validation set can also make that set part of the design process. A separate test set is needed for an independent final assessment.

Our setting. `val_dataset` defaults to a fixed set of 1,000 DCLM rows. All its target tokens contribute to each evaluation, including a final partial batch. `num_evals=100` requests approximately 100 evaluations over the run; evaluation also occurs at the final step. The interval is $\lfloor S/\text{num_evals} \rfloor + 1$ optimizer steps for a run of S steps. Changing evaluation frequency does not change the gradient objective, but changes monitoring resolution and runtime.

2 Data

2.1 Tokenization, Vocabulary, and Special Tokens

A tokenizer maps text to a sequence of integer IDs and provides a way to decode those IDs back into text. A token can represent a word, part of a word, punctuation, whitespace, or another text fragment. Token

IDs are categorical labels: adjacent IDs need not have similar meanings. The vocabulary is the finite set of possible tokens; its size V determines how many possible next-token outcomes the model scores.

Tokenizers also reserve IDs for special tokens. A beginning-of-sequence token (BOS) can mark a document's start; an end-of-sequence token (EOS) can mark its end. A padding token, when used, fills otherwise empty positions in a batch. These symbols are conventions supplied by data preparation, not mathematical requirements of a transformer. A BOS token inside a packed sequence still has an ordinary learned embedding. A boundary marker does not automatically prevent attention across the boundary.

Our setting. Our prepared data uses a SentencePiece tokenizer with $V = 4096$, including its special tokens. Tokenization happens before training; the trainer reads integer IDs. Changing the tokenizer requires rebuilding the data and matching the model's `vocab_size`. Training does not insert BOS or EOS tokens on the fly, or reset attention merely because one occurs in a row.

2.2 Concatenation, Chunking, and Context Length

Documents have different lengths, but dense training batches are convenient when rows have a common length T . One preparation strategy tokenizes documents, includes boundary markers, concatenates token streams, and chunks them into fixed-length rows. A row can then contain pieces of multiple documents, and a long document can span multiple rows. Chunk boundaries and document boundaries are not necessarily the same.

The *context length* limits the input available within one forward pass. Predictions near a row's beginning have less preceding context than those near its end. Independent-row training does not carry hidden state from one row to the next. Increasing context length changes both the prediction problem and computation: ordinary attention involves pairs of positions, so its arithmetic grows quadratically with sequence length.

Our setting. The files contain already prepared rows of 1,024 tokens. We train on these rows as provided, with causal attention inside each row and no state carried between rows. The dataset and architecture `context_length` must agree. Changing a model field alone does not repack the binary data. The default dataset uses fixed-length rows without padding.

2.3 Dataset Size, Subsets, and Storage

With N rows of length T , the input-token count is NT , while the number of within-row next-token targets is $N(T - 1)$. These are related but not identical notions of a training budget. The total amount processed also depends on how many passes training makes over the selected data.

Preprocessing text into a rectangular array of integer token IDs avoids repeating tokenization during training. A binary file can be memory-mapped: the program accesses slices of a file-backed array without first reading the entire dataset into an ordinary in-memory array. The storage format should not change which examples or targets define the loss.

Our setting. `train_dataset` defaults to the DCLM training pool of 9,600,000 rows. `num_train_sequences=600_000` selects 600,000 rows, or 614,400,000 input tokens and 613,800,000 targets per complete pass. Train and validation tokens are stored in binary arrays with metadata recording shape and integer type. Custom dataset specifications can supply a different path, name/tag, context length, and sequence limit; the tokens must match the model's vocabulary.

2.4 Ordering and Data Seeds

A data seed makes random shuffling reproducible. When training uses only a subset of the data, changing the seed can change both the selected examples and their order.

Our setting. `data_seed=42` globally shuffles the training pool before taking the requested N rows. Each epoch reuses this order; validation is unchanged.

3 Optimization

3.1 Minibatches and Gradient Descent

A gradient contains the partial derivative of the loss with respect to each trainable parameter. Gradient descent updates parameters in the negative gradient direction. A minibatch $\mathcal{B}_s$ estimates the population loss at update s :

$$\begin{aligned}\widehat{L}_s(\theta) &= \frac{1}{|\mathcal{B}_s|} \sum_{x \in \mathcal{B}_s} \ell(\theta; x), && \text{batch estimate of the objective,} \\ g_s &= \nabla_{\theta} \widehat{L}_s(\theta_s), && \text{batch gradient,} \\ \theta_{s+1} &= \theta_s - \alpha_s g_s, && \text{plain gradient-descent update.}\end{aligned}$$

Here α_s is the learning rate. Using sampled minibatches gives stochastic gradient descent (SGD).

Our setting. `batch_size=64` is the number of sequences per optimizer update across the available GPUs. Each full default update uses 65,536 input tokens. The default optimizer is AdamW rather than the plain update above, but it uses the same minibatch gradient as its input.

3.2 Backpropagation and Gradient Accumulation

A layer computes $h^{(\ell)} = f_{\ell}(h^{(\ell-1)}; \theta^{(\ell)})$, where $\theta^{(\ell)}$ contains its trainable weights. Given the loss derivatives with respect to this layer's outputs, the chain rule gives

$$\begin{aligned}\frac{\partial L}{\partial \theta_i^{(\ell)}} &= \sum_j \frac{\partial L}{\partial h_j^{(\ell)}} \frac{\partial h_j^{(\ell)}}{\partial \theta_i^{(\ell)}}, && \text{gradient for each parameter,} \\ \frac{\partial L}{\partial h_i^{(\ell-1)}} &= \sum_j \frac{\partial L}{\partial h_j^{(\ell)}} \frac{\partial h_j^{(\ell)}}{\partial h_i^{(\ell-1)}}, && \text{gradient passed to the previous layer.}\end{aligned}$$

Backpropagation starts at the loss and repeats these computations backward through the layers. The parameter derivatives form the gradient used by the optimizer. PyTorch's *autograd* records the computation graph during the forward pass; `loss.backward()` automatically computes and accumulates these parameter gradients.

A large logical batch can be split into M microbatches. For equal-sized microbatches, the full-batch mean gradient is $g_s = M^{-1} \sum_{j=1}^M \nabla_{\theta} \widehat{L}_{s,j}(\theta_s)$. We can compute each term in succession and accumulate parameter gradients, discarding its intermediate activations before the next forward pass. Parameters must remain unchanged until all microbatches have contributed. Clipping and the optimizer update then happen once, not once per microbatch. The gradients are cleared before the next logical update.

Our setting. `num_micro_batches=1` uses the whole batch at once. Larger values must divide `batch_size`; the loss of each microbatch is divided by this count before backpropagation. Accumulation lowers peak activation memory, but is not necessarily faster or bitwise equivalent to a single large batch. Use divisible dataset/batch sizes for clean comparisons; partial microbatches need not give the same sample weighting. Our trainer disables compilation when more than one microbatch is requested.

3.3 Epochs and the Training Budget

An epoch is one pass over the selected training dataset. For N examples and full batches of size B , one epoch uses N/B updates when B divides N . Multiple epochs revisit examples; a fractional epoch ends before a complete pass. Increasing the number of epochs and increasing the number of distinct examples are different interventions even if the token counts match.

The training budget determines the duration over which a learning-rate schedule runs. Thus changing epochs or batch size can change both how often data is seen and the learning rate at a particular token count. When comparing runs, specify whether the comparison is at equal updates, tokens, or wall-clock time.

Our setting. `num_epochs=1.0` and 600,000 rows at batch size 64 give exactly 9,375 optimizer steps. The code uses $S = \lfloor \text{num_epochs} \cdot \lfloor K/M \rfloor \rfloor$, where K is the number of microbatches (including a possible partial one) and $M = \text{num_micro_batches}$. The default sizes divide evenly, so partial-batch conventions do not affect the default run. Training stops at this budget, not automatically when validation loss stops improving.

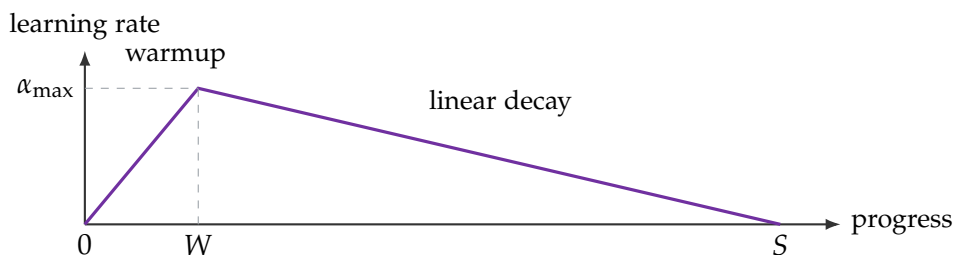
3.4 Learning Rate and Scheduling

The learning rate scales each optimizer update. If it is small, parameters move slowly; a sufficiently large value can cause unstable updates. A schedule specifies this scale as a function of training progress. *Warmup* starts with small updates and increases their scale early in training. *Decay* reduces it later in the run.

For S planned updates, W warmup steps, peak learning rate $\alpha_{\max}$, and schedule index s , linear warmup followed by linear decay is

$$\alpha_s = \alpha_{\max} \begin{cases} s/W, & 0 \leq s < W, \\ (S - s)/(S - W), & W \leq s \leq S. \end{cases}$$

The warmup branch is omitted when $W = 0$. After warmup, let $u = (s - W)/(S - W)$. Cosine decay uses $\alpha_s = \alpha_{\max}(1 + \cos(\pi u))/2$ instead. A constant schedule keeps the learning rate fixed. A warmup-stable-decay schedule keeps the peak value for a middle phase and linearly decays during a chosen final fraction.



Schematic: warmup is widened for visibility.

Our setting. `learning_rate=0.003`, `lr_schedule="linear"`, and `warmup_percent=0.01` specify the peak rate, schedule, and warmup fraction. Warmup lasts $\lfloor 0.01S \rfloor = 93$ steps in the default run. The scheduler starts at index zero: with warmup, the first update uses zero learning rate. It advances after each optimizer update. Other supported choices are "cos", "constant" (no warmup), and, for example, "wsd0.1" (linear decay over the final 10% of steps).

3.5 SGD with Momentum

Momentum keeps a running direction from past gradients. A common SGD convention, with buffer b_s , is

$$b_s = \mu b_{s-1} + g_s, \quad \theta_{s+1} = \theta_s - \alpha_s b_s, \quad b_0 = 0,$$

where updates in this expression are indexed starting at 1. The momentum coefficient μ controls how long earlier gradients contribute. With $\mu = 0$, this reduces to plain SGD. This buffer uses a sum, not the $(1 - \mu)g_s$ averaging convention used in Adam below; learning-rate interpretations depend on that convention.

Our setting. `optimizer_name="sgd"` selects SGD with momentum $\mu = \text{beta1}$. Its default value is 0.9; setting `beta1=0` gives plain SGD. There is no Nesterov momentum or dampening, and `beta2` is unused for SGD. For decayed parameters, our SGD adds $\lambda\theta_s$ to the gradient before updating its momentum buffer. This is coupled L2 regularization, unlike AdamW's separate decay step.

3.6 Adam and Adaptive Updates

Adam maintains two moving averages for each parameter: a first moment m_s of gradients and a second moment v_s of squared gradients. For this subsection, number updates from $s = 1$, start with $m_0 = v_0 = 0$, and let g_s be the gradient supplied to the optimizer (after clipping, if enabled):

$$\begin{aligned} m_s &= \beta_1 m_{s-1} + (1 - \beta_1) g_s, & \text{average gradient,} \\ v_s &= \beta_2 v_{s-1} + (1 - \beta_2) (g_s \odot g_s), & \text{average squared gradient,} \\ \hat{m}_s &= m_s / (1 - \beta_1^s), \quad \hat{v}_s = v_s / (1 - \beta_2^s), & \text{initialization-bias correction,} \\ \theta_{s+1} &= \theta_s - \alpha_s \frac{\hat{m}_s}{\sqrt{\hat{v}_s + 10^{-8}}}, & \text{update before weight decay.} \end{aligned}$$

Products, squares, square roots, and divisions are coordinatewise. The second moment rescales coordinates according to recent gradient magnitude; it is not a centered variance estimate. Bias correction compensates for starting the moving averages at zero. The fixed 10^{-8} added to the denominator is for numerical stability when its other term is very small.

Larger β_1 or β_2 gives the corresponding moving average a longer memory. Adam needs optimizer state in addition to model weights: two moment tensors plus update counters. Identical weights with different moment histories can produce different next updates.

Our setting. The default `optimizer_name="adamw"` uses Adam's adaptive update with `beta1=0.9` and `beta2=0.95`, plus decoupled weight decay. Numerical stabilization is fixed at 10^{-8} , not a configuration choice. CUDA runs use PyTorch's fused AdamW implementation.

3.7 Weight Decay and Parameter Groups

Weight decay shrinks selected parameters toward zero. In AdamW it is *decoupled* from the gradient's moving averages. For a decayed parameter, the combined update is

$$\theta_{s+1} = (1 - \alpha_s \lambda) \theta_s - \alpha_s \frac{\hat{m}_s}{\sqrt{\hat{v}_s + 10^{-8}}}.$$

Here λ is the weight-decay coefficient. Decay therefore depends on both its coefficient and the learning-rate schedule. With adaptive optimizers, inserting $\lambda\theta$ into the gradient would also affect the moment estimates and is not the same rule.

Parameters can be divided into groups with different decay settings. This lets a recipe decay large projection matrices while leaving embeddings or normalization scales unchanged by the decay operation. Exclusion from decay does *not* freeze a parameter: it still receives gradient updates.

Our setting. `weight_decay=0.1`. Decay is always masked: embedding parameters, normalization parameters, and biases are excluded. The untied output projection is a linear-layer weight and is decayed; when tied to the input embedding, that shared parameter is excluded. Attention and MLP projection weights are decayed. This masking policy is fixed, not an extra flag.

3.8 Gradient Clipping

Occasionally a batch can yield a large gradient. Global norm clipping concatenates all parameter gradients conceptually and limits their joint Euclidean norm. With threshold c , it replaces the gradient by

$$g_s^{\text{clip}} = g_s \min\left(1, \frac{c}{\|g_s\|_2}\right),$$

with the factor taken as 1 for a zero gradient. Clipping preserves the gradient direction while reducing its magnitude when the threshold is exceeded. It differs from clipping each component separately.

The optimizer then processes the clipped gradient. For Adam, limiting the gradient norm does not directly impose the same limit on the parameter-update norm, since moments and coordinatewise rescaling also affect the update. Weight decay is a separate contribution and is not bounded by this operation.

Our setting. `grad_norm=1.0` clips the global parameter-gradient norm after all microbatches have accumulated and before the optimizer step. Set `grad_norm=None` to disable it. A diagnostic collected immediately after backward sees the pre-clipping gradient, which can exceed 1.0.

3.9 Numerical Precision and Execution

Floating-point arithmetic stores approximations to real numbers. `fp32` uses 32 bits; `bfloat16` (`bf16`) uses 16, with a similar exponent range but fewer significant digits. Lower precision reduces memory and can accelerate matrix multiplication on suitable hardware, but changes rounding error. The dtype of parameters, intermediate computations, gradient storage, and optimizer state need not all be the same.

Mixed precision keeps some state in `fp32` while running selected operations in lower precision. Auto-casting chooses operation-specific dtypes; it does not simply cast the entire model and all reductions to `bf16`. Storing parameters and moments in `fp32` helps retain small updates, though it cannot undo errors introduced in earlier lower-precision computations.

Compilation and fused kernels combine operations or reduce dispatch overhead. They aim to preserve the mathematical computation but can change floating-point evaluation order. Likewise, splitting a batch across GPUs changes where computations and reductions occur. Neither option defines a new loss function.

Our setting. `precision="mp"` stores parameters, their gradients, and Adam state in fp32, with bf16 autocast on CUDA. `"fp32"` disables that autocast; `"bf16"` stores parameters in bf16 as well and is a different numerical recipe. The loss is evaluated in fp32 in all modes; RMSNorm's mean-square calculation and RoPE trigonometry also use fp32. We do not use a gradient scaler in the default bf16 mixed-precision path.

`torch_compile_mode="reduce-overhead"` selects the compilation mode for the training loss computation on one GPU with one microbatch. Multiple GPUs use data parallelism without this compiled path. The execution mode affects memory and runtime; a speed comparison should include startup and evaluation costs as well as optimizer-step time.

3.10 Random Seeds and Determinism

A pseudorandom generator produces a reproducible stream from a seed. Random initialization, dropout, and dataset permutation can all influence an experiment. Controlling seeds makes comparisons easier to interpret, but a seed alone is not a guarantee of bitwise-identical GPU computations. Some parallel algorithms have nondeterministic reduction orders.

Deterministic execution restricts the algorithms used to avoid known sources of nondeterminism. It may be slower or reject an operation with no supported deterministic implementation. Reproducibility across different devices, library versions, or numerical precision still requires care.

Our setting. `model_seed=42` seeds initialization and model-side randomness; `data_seed=42` separately chooses training rows and their order. `model_seed=None` skips explicit model seeding. The default `deterministic=False` does not enforce deterministic algorithms. Enabling it requests deterministic PyTorch behavior, configures deterministic CUDA matrix multiplication, and disables TF32 and cuDNN benchmarking. It does not promise identical results across hardware or software versions.

3.11 Diagnostics and Experiment Records

Loss alone does not describe the internal state of optimization. Gradient norms measure the size of the signal passed to the optimizer. Parameter and activation magnitudes reveal how scales evolve across modules. A common scale statistic for a tensor with n entries is its root mean square (RMS), $\text{RMS}(a) = \sqrt{n^{-1} \sum_i a_i^2}$. Unlike the Euclidean norm, RMS divides by the number of entries, making it less directly dependent on tensor size. It is not the standard deviation unless the mean is zero.

The time at which a metric is measured matters. Pre-clipping gradients, post-update parameters, and evaluation-mode activations describe different states. A metric may also require extra forward or backward computations; logging is not necessarily free. Observational metrics should not update the model or unintentionally change subsequent training randomness.

Our setting. `metric_loggers` attaches Python diagnostics after backward, after an optimizer step, or after evaluation. They receive the current model, optimizer, scheduler, batch, loss, step, configuration, and validation batches; returned scalars are logged under `logging/`. The current

default includes per-module and global activation, parameter, and pre-clipping gradient RMS on zero-based steps 0, 100, 200, and so on. The global activation statistic pools recorded parameterized leaf-module outputs, not a single hidden state.

`wandb_online=True` enables W&B records and these custom loggers. `wandb_entity` and `wandb_project` choose the destination; `wandb_tags` and `run_name_suffix` organize experiments rather than change the ML computation. Training loss is recorded per optimizer step and validation loss at evaluation points. A logged training loss is measured before that step's update, while its validation loss uses the updated model.

3.12 Initialization from Checkpoints and Resuming

Loading model weights supplies a starting point for a new experiment. Resuming training also restores the optimizer's history, such as Adam's moment estimates. These give different subsequent updates even when the starting weights are identical.

Our setting. `init_checkpoint_path` loads model weights, but starts a new optimizer and learning-rate schedule. `resume_from_checkpoint=True` resumes an existing run from latest .pt, restoring weights, optimizer and random state, and the schedule and data positions. Continuing the same trajectory requires keeping the data and recipe unchanged. `force_run=True` bypasses reuse of a completed model; it does not turn off checkpoint resumption. For a fresh rerun, also set `resume_from_checkpoint=False` and leave `init_checkpoint_path=None`.

4 Architecture

4.1 Tensors, Dimensions, and Model Size

A neural network composes parameterized operations. Its *architecture* specifies their structure and dimensions; training chooses parameter values within that structure. A tensor is a multidimensional array. For this handout, use the following dimension names:

Symbol	Meaning
B	sequences in a forward-pass batch
T	token positions per sequence
V	vocabulary size
L	transformer blocks (depth)
d	hidden or residual-stream width
H, d_h	attention heads and dimensions per head, with $d = Hd_h$
d_{ff}	MLP intermediate width

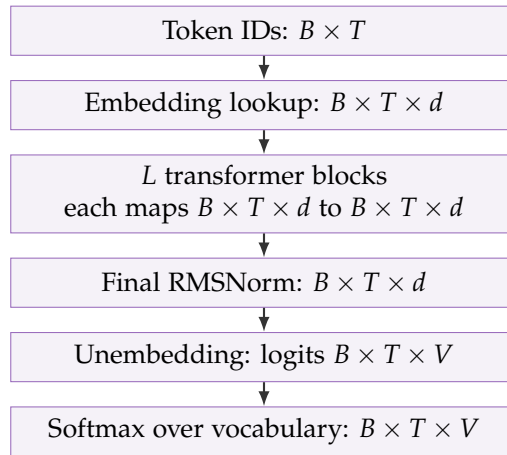
The batch axis indexes independent sequences. A token representation has d coordinates; these are learned features rather than vocabulary probabilities. Depth and width change the family of representable functions and the cost of computing them.

Our setting. `model_name="d8"` chooses $L = 8$, $d = 512$, $H = 8$, $d_h = 64$, and $d_{ff} = 1792$. Thus d8 has eight attention modules *and* eight MLP modules. Section 4.14 describes how the other dimensions change along our model-depth ladder.

`model_config` can supply a custom `LMConfig` with a descriptive name. Its `vocab_size` sets V , `context_length` sets T , `hidden_size` sets d , `intermediate_size` sets d_{ff} , and `num_hidden_layers` sets L . `num_attention_heads` and `head_dim` set H and d_h , with $d = Hd_h$. Key/value head count and the position limit are discussed below. The preset ladder is a convention, not a requirement that these choices always change together.

4.2 Overall Transformer Computation

The model first looks up a vector for each input token. A stack of transformer blocks then updates these vectors. A final normalization and output projection convert them into scores for possible next tokens. Each block combines attention, which mixes information across positions, and a multilayer perceptron (MLP), which applies nonlinear computation independently at each position. This is a decoder-only model: it predicts from the preceding token context without a separate encoder network or cross-attention to encoder outputs.



With input IDs $X \in \{0, \dots, V-1\}^{B \times T}$, embedding table $E \in \mathbb{R}^{V \times d}$, and output matrix $W_U \in \mathbb{R}^{d \times V}$, the forward pass is

$$\begin{aligned}
 H^{(0)} &= E[X] \in \mathbb{R}^{B \times T \times d}, & \text{token vectors,} \\
 H^{(\ell)} &= \text{Block}_\ell(H^{(\ell-1)}) \in \mathbb{R}^{B \times T \times d}, & \ell = 1, \dots, L, \\
 \tilde{H} &= \text{RMSNorm}(H^{(L)}) \in \mathbb{R}^{B \times T \times d}, & \text{final normalization,} \\
 Z &= \tilde{H}W_U \in \mathbb{R}^{B \times T \times V}, & \text{vocabulary scores,} \\
 P &= \text{softmax}_V(Z) \in \mathbb{R}^{B \times T \times V}, & \text{next-token probabilities.}
 \end{aligned}$$

Our setting. The native architecture is a decoder-only, pre-normalized transformer with independently parameterized blocks. It returns logits; the loss computation forms log probabilities from them. `model_builder=None` uses this model. A custom `model_builder` and `model_builder_kwargs` can supply an experiment-specific architecture, provided it follows the training and saving interfaces. These do not change the default model automatically.

4.3 Embedding Lookup

An embedding table contains one learned vector per vocabulary item. For token ID v , lookup returns row $E_v \in \mathbb{R}^d$. The same table is used at every position. Lookup is equivalent to multiplying a one-hot vector by the table, but does not require constructing the mostly-zero one-hot array. Learning can make token vectors useful for prediction without assigning an interpretation to each individual coordinate.

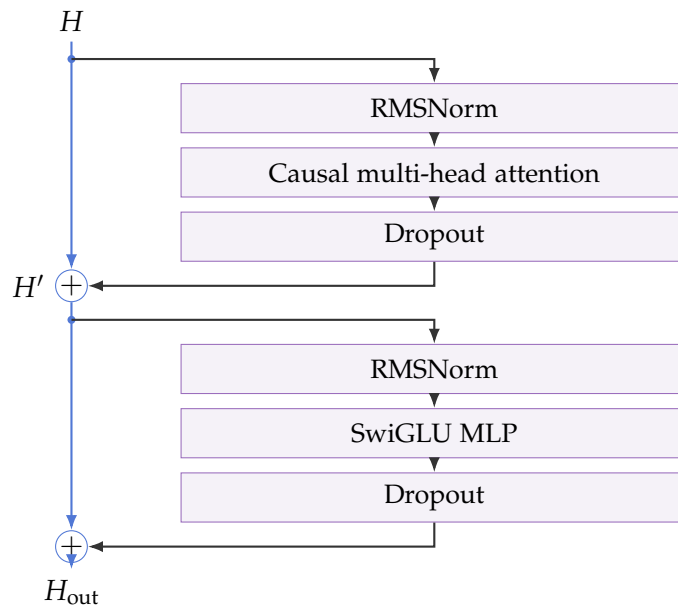
Our setting. The default embedding table has shape 4096×512 . We do not multiply its output by an extra $\sqrt{d}$ factor or add learned absolute-position vectors.

4.4 Residual Connections and Pre-Normalization

A residual connection adds a branch's output to its input: $h \mapsto h + f(h)$. The running hidden representation is called the *residual stream*. The branch learns a change to that stream rather than replacing it entirely. Addition also gives gradients a direct path through the identity contribution, alongside the path through the branch. Residual connections do not guarantee stable optimization, but change how information and gradients propagate.

In a *pre-norm* block, each branch receives a normalized version of the residual stream. Normalization is inside the branch; the skip path preserves the unnormalized stream. This differs from normalizing the result of the addition (post-norm). With $H \in \mathbb{R}^{B \times T \times d}$, one block is

$$\begin{aligned}
 Y &= \text{RMSNorm}_{\text{attn}}(H), & Y &\in \mathbb{R}^{B \times T \times d}, \\
 A &= \text{Attention}(Y), & A &\in \mathbb{R}^{B \times T \times d}, \\
 H' &= H + \text{Dropout}(A), & H' &\in \mathbb{R}^{B \times T \times d}, \\
 U &= \text{RMSNorm}_{\text{mlp}}(H'), & U &\in \mathbb{R}^{B \times T \times d}, \\
 F &= \text{MLP}(U), & F &\in \mathbb{R}^{B \times T \times d}, \\
 H_{\text{out}} &= H' + \text{Dropout}(F), & H_{\text{out}} &\in \mathbb{R}^{B \times T \times d}.
 \end{aligned}$$



Inside *one* of the L blocks in the previous diagram. Every stream and branch output has shape $B \times T \times d$. The straight blue path is the residual stream; each branch adds its output back into that stream.

Our setting. Each default block has two residual additions: one around attention and one around the MLP. Both use pre-norm, with separate learned RMSNorm scales. There is no depth-dependent multiplier on the skip paths. A separate final RMSNorm follows the entire stack before unembedding.

4.5 RMS Normalization

Normalization controls the scale seen by later operations. For a token vector $h \in \mathbb{R}^d$, RMSNorm computes

$$\text{RMSNorm}(h)_i = \gamma_i \frac{h_i}{\sqrt{d^{-1} \sum_{j=1}^d h_j^2 + 10^{-5}}}, \quad i = 1, \dots, d.$$

The learned vector $\gamma \in \mathbb{R}^d$ scales individual coordinates; the small constant stabilizes division for near-zero inputs. RMSNorm does not subtract the mean and has no learned additive offset here. It normalizes each token separately over its feature coordinates, not over the batch or sequence. Consequently it has no running batch statistics to estimate or freeze at evaluation time.

Normalization does not force every later activation to have unit RMS: learned scales, subsequent projections, and residual additions can all change its magnitude. In pre-norm, it is specifically the input to a branch that is normalized, not the entire residual stream after every operation.

Our setting. Every block uses two width- d RMSNorm modules and the model uses a final one. Their scales start at 1 and are trained without weight decay. The stabilizer is fixed at 10^{-5} . Query/key normalization uses the same operation over d_h coordinates, as described below.

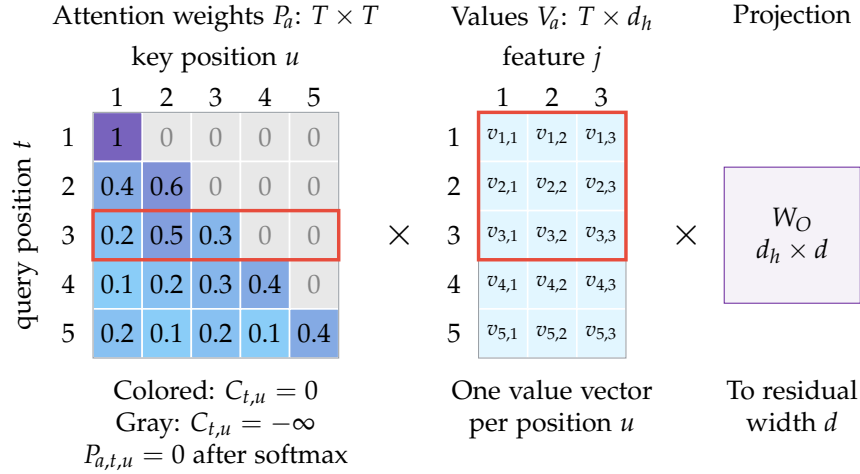
4.6 Single-Head Causal Attention

Self-attention allows each position to read information from earlier positions in the same sequence. A *query* specifies what a position reads, a *key* participates in determining how much it is read, and a *value* is the vector being mixed. These are all learned projections of the current token representations, not externally supplied labels.

Consider one sequence, omitting the batch axis. Its representations form $Y \in \mathbb{R}^{T \times d}$. A single attention head uses three learned matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}$, where d_h is the head width. A learned output projection $W_O \in \mathbb{R}^{d_h \times d}$ maps its weighted mixture back to residual-stream width. The computation is

$$\begin{aligned} Q &= YW_Q \in \mathbb{R}^{T \times d_h}, & \text{queries,} \\ K &= YW_K \in \mathbb{R}^{T \times d_h}, & \text{keys,} \\ V_a &= YW_V \in \mathbb{R}^{T \times d_h}, & \text{values,} \\ S &= QK^\top / \sqrt{d_h} + C \in \mathbb{R}^{T \times T}, & \text{masked similarity scores,} \\ P_a &= \text{softmax}_{\text{keys}}(S), & \text{weights over source positions,} \\ O &= P_a V_a \in \mathbb{R}^{T \times d_h}, & \text{weighted mixtures of values,} \\ \text{Attention}(Y) &= OW_O \in \mathbb{R}^{T \times d}, & \text{project back to residual-stream width.} \end{aligned}$$

The score $S_{t,u}$ compares the query at position t with the key at position u . The causal mask has entries $C_{t,u} = 0$ for $u \leq t$ and $-\infty$ for $u > t$. Softmax turns each row of scores into nonnegative weights summing to one. Thus the intermediate mixture at position t is a weighted average of the available value vectors: $O_t = \sum_{u \leq t} P_{a,t,u} V_{a,u}$. The attention output at that position is $O_t W_O$. Position t can read itself: its output predicts token $t + 1$, so this does not reveal the prediction target.



Example with $T = 5$ and $d_h = 3$; weights are illustrative. The red outlines highlight query $t = 3$ and the value rows it can read: $O_3 = 0.2V_{a,1} + 0.5V_{a,2} + 0.3V_{a,3}$. The attention output is O_3W_O . Future positions receive zero attention weight.

Dividing dot products by $\sqrt{d_h}$ controls their scale as head width changes. Very concentrated softmax weights and nearly uniform weights produce different mixtures of values.

Our setting. Each of our heads uses this causal computation with $d_h = 64$. Before forming scores, we apply RoPE and then optional QK normalization to the queries and keys, as described below. Attention dropout, when enabled, acts on P_a before multiplication by the values; it defaults to zero.

4.7 Multi-Head Attention

Multi-head attention runs H such heads on the same input Y , each with its own query, key, and value projections.¹ Each head can form a different pattern of attention over positions. Their mixtures $O^{(h)}$, before output projection, are concatenated along the feature axis and mixed by one learned output projection $W_O \in \mathbb{R}^{Hd_h \times d}$:

$$\begin{aligned}
 O^{(h)} &= \text{Head}_h(Y) \in \mathbb{R}^{T \times d_h}, & h &= 1, \dots, H, \\
 O_{\text{concat}} &= [O^{(1)} \dots O^{(H)}] \in \mathbb{R}^{T \times Hd_h}, & & \text{concatenate head outputs,} \\
 \text{Attention}(Y) &= O_{\text{concat}} W_O \in \mathbb{R}^{T \times d}, & & \text{return to residual-stream width.}
 \end{aligned}$$

Heads are computed in parallel, not sequentially. With a batch of B sequences, queries, keys, and values can be arranged as $B \times H \times T \times d_h$; attention weights have shape $B \times H \times T \times T$. The combined output has shape $B \times T \times d$. The matrix W_O mixes features from different heads; it is distinct from the unembedding matrix that predicts vocabulary IDs.

Our setting. d8 uses $H = 8$ heads of width $d_h = 64$, with $Hd_h = d = 512$. The per-head query, key, and value matrices are stored together as three 512×512 projections; W_O is also 512×512 . Computation uses scaled dot-product attention (SDPA), which can avoid materializing the full attention matrix with suitable GPU kernels. The equations describe the operation, not a requirement to store all $T \times T$ scores explicitly.

¹Grouped-query attention shares fewer key/value heads among groups of query heads. It is off by default in our models: d8 uses eight query heads and eight key/value heads. A custom configuration can reduce `num_key_value_heads`, provided it divides `num_attention_heads`.

4.8 Rotary Position Embeddings

Token vectors alone do not explicitly encode distances between positions. Rotary position embeddings (RoPE) add this information by rotating pairs of query and key coordinates, rather than adding a positional vector to the residual stream. For position t , a coordinate pair is transformed by

$$R(t\omega_i) = \begin{pmatrix} \cos(t\omega_i) & -\sin(t\omega_i) \\ \sin(t\omega_i) & \cos(t\omega_i) \end{pmatrix}, \quad \omega_i = \theta_{\text{RoPE}}^{-2i/d_h}, \quad i = 0, \dots, d_h/2 - 1.$$

Let $q_t = (YW_Q)_t$ and $k_t = (YW_K)_t$ be the projected query and key at position t , each with d_h coordinates. Using zero-based coordinate indices and pairing the two halves of each vector, the rotations are applied as

$$\begin{pmatrix} \tilde{q}_{t,i} \\ \tilde{q}_{t,i+d_h/2} \end{pmatrix} = R(t\omega_i) \begin{pmatrix} q_{t,i} \\ q_{t,i+d_h/2} \end{pmatrix}, \quad \text{rotate each query pair,}$$

$$\begin{pmatrix} \tilde{k}_{t,i} \\ \tilde{k}_{t,i+d_h/2} \end{pmatrix} = R(t\omega_i) \begin{pmatrix} k_{t,i} \\ k_{t,i+d_h/2} \end{pmatrix}, \quad \text{rotate each key pair.}$$

Repeating this for every pair gives the rotated vectors $\tilde{q}_t, \tilde{k}_t \in \mathbb{R}^{d_h}$. These replace the original queries and keys in attention (with QK normalization applied next when enabled).

Different pairs rotate at different frequencies. Since $R(t\omega_i)^\top R(u\omega_i) = R((u-t)\omega_i)$, the dot product of rotated queries and keys incorporates the relative offset $u-t$. Content and position both influence attention. Values are not rotated.

The RoPE frequency base determines the range of these frequencies. The largest supported position index is distinct from the training context length: allowing a longer input in code does not imply the model has learned to use such lengths reliably.

Our setting. RoPE uses a fixed base $\theta_{\text{RoPE}} = 500,000$, with trigonometry computed in fp32 and then cast for the attention computation. It pairs coordinates across the two halves of each head vector. Positions restart at zero for each row. `max_position_embeddings=131_072` is the architecture's length limit; actual default training context remains 1,024. There are no trainable positional-embedding parameters.

4.9 Query/Key Normalization

The magnitudes of queries and keys affect attention scores, not just their directions. Query/key normalization (QK norm) applies RMSNorm separately to each head's query and key vector before the scaled dot product. This reduces the direct dependence of scores on the incoming vector norms. Learned scale parameters still allow the score scale to change during training. Values are not normalized by this operation.

Our setting. `qk_norm=True` applies separate learned query and key RMSNorm scales after RoPE, over each head's 64 coordinates. Within a block the scales are shared across heads, not across the query and key modules; the stabilizer is 10^{-5} . With `qk_norm=False`, those normalizations are omitted.

4.10 The SwiGLU MLP

The MLP transforms each token independently with weights shared over positions. It changes features within a token; unlike attention, it does not mix positions. Expanding to an intermediate width allows nonlinear combinations of features before returning to residual-stream width. Our gated MLP has three matrices: $W_g, W_u \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $W_d \in \mathbb{R}^{d_{\text{ff}} \times d}$. Its operations are

$$\begin{aligned} G &= YW_g, & U &= YW_u, & G, U &\in \mathbb{R}^{B \times T \times d_{\text{ff}}}, \\ A &= \text{SiLU}(G) \odot U, & & & A &\in \mathbb{R}^{B \times T \times d_{\text{ff}}}, \\ \text{MLP}(Y) &= AW_d, & & & \text{MLP}(Y) &\in \mathbb{R}^{B \times T \times d}. \end{aligned}$$

Here $\text{SiLU}(z) = z \sigma(z)$, with $\sigma(z) = 1/(1 + e^{-z})$, applied elementwise. The nonlinear gate modulates a second projected representation through the elementwise product $\odot$. This combination is called SwiGLU. Without nonlinearities or gating, a sequence of linear maps would collapse to a single linear map.

Our setting. d8 uses $d = 512$ and $d_{\text{ff}} = 1792$. All three projections are bias-free. The expansion ratio is 3.5 in the preset family, but `intermediate_size` can change it in a custom model configuration. There is no extra dropout inside the gate; residual-branch dropout is applied after the down projection.

4.11 Unembedding, Softmax, and Weight Tying

After the final RMSNorm, the unembedding maps each hidden vector to V logits, one per vocabulary item. For $\tilde{h}_t \in \mathbb{R}^d$ and $W_U \in \mathbb{R}^{d \times V}$, write $z_t = \tilde{h}_t W_U \in \mathbb{R}^V$. Softmax converts these unconstrained real scores into probabilities:

$$p_\theta(x_{t+1} = v \mid x_{\leq t}) = \frac{e^{z_{t,v}}}{\sum_{u=0}^{V-1} e^{z_{t,u}}}.$$

Only differences between logits matter: adding a common constant leaves probabilities unchanged. Numerically, subtracting a common maximum before exponentiation avoids unnecessarily large exponentials. Log-softmax computes log probabilities directly for the cross-entropy loss.

With *weight tying*, the output matrix is constrained to be $W_U = E^\top$, where E is the input embedding table. There is one shared trainable tensor, not two tensors periodically copied to each other. Gradients from its input and output roles accumulate into the same parameter. Tying reduces the number of independent parameters and constrains how input representations and output scores relate.

Our setting. `tie_word_embeddings=False` keeps separate input and output parameters. Setting it to True shares them. Untied d8 has a 4096×512 input table and an independent 512×4096 mathematical output matrix. The output projection has no bias. As discussed under parameter initialization, separate parameters can still start from correlated initial values; that is not weight tying.

4.12 Dropout and Training versus Evaluation

Dropout randomly removes entries of an activation during training. For probability $p < 1$, independent masks $m_i \sim \text{Bernoulli}(1 - p)$ produce $\text{Dropout}(a)_i = m_i a_i / (1 - p)$. This scaling preserves each entry's conditional expectation. At evaluation, dropout is the identity; no random mask or rescaling is applied. Preserving expectation at one layer does not make a nonlinear network's training output equal to its evaluation output.

Attention dropout removes some attention-weight contributions after softmax. The surviving weights are scaled, not renormalized to sum to one. Residual dropout instead acts on a completed attention or MLP branch before it is added to the residual stream. The identity skip path remains intact. These are different locations even if their probabilities are the same.

Our setting. `dropout=0.0` disables dropout by default. A nonzero value governs all three uses in each block: attention weights, attention residual output, and MLP residual output. There is no embedding dropout. Evaluation switches all of them off; consequently training and validation losses at nonzero dropout differ in both data and stochastic computation.

4.13 Parameter Initialization

Initialization selects the starting point for optimization. Random weights break symmetries between features, while their scale influences initial activations and gradients. For a projection summing d_{in} roughly independent contributions, scaling weight magnitudes like $1/\sqrt{d_{\text{in}}}$ helps keep output scales from growing simply because the input width increases. This is a scale argument, not a guarantee about a complete network's dynamics.

A truncated normal draws from a normal distribution conditioned to lie within chosen bounds. Specifying a normal's standard deviation and then truncating its tails makes the resulting standard deviation slightly smaller. Normalization scales and embedding matrices need not follow the same initialization rule as other projections.

Our setting. For attention and MLP matrices, entries are initialized from a mean-zero normal with nominal standard deviation $1/\sqrt{d_{\text{in}}}$, truncated at three nominal standard deviations. All RMSNorm scales start at 1.

For embeddings, draw one base table $A \in \mathbb{R}^{V \times d}$ from a standard normal truncated to $[-3, 3]$. In the default untied case,

$$E = A/d, \quad W_U = A^\top / \sqrt{d}.$$

The tables therefore begin with aligned directions but are independently updated afterward. In the tied case, the shared table instead starts at $E = A/\sqrt{d}$ and $W_U = E^\top$. No default projection is initialized to zero, and no additional depth-dependent scaling is applied to residual branch weights. `model_seed` controls these draws; initialization scale is fixed by the recipe rather than exposed as another `TrainConfig` field.

4.14 Scaling Model Depth

Depth is the number of transformer blocks applied in sequence. Increasing depth alone adds more attention and MLP computations while keeping each block's dimensions fixed. The additional blocks have their own trainable parameters; they are not repeated applications of the same weights.

A model family can instead increase depth and width together. Wider residual streams require wider embeddings, attention projections, MLPs, and normalization scales. Attention can grow by adding heads, increasing the dimension of each head, or both. These choices define different scaling experiments. More blocks do not increase the available context length: causal attention can already read all preceding positions within a row.

Our setting. Our `model_name` presets include every integer depth from d4 through d18. The name d8 is shorthand for a complete size configuration, not just a block count. For preset dL, the dimensions are:

Component	Rule	d4	d8	d12
Transformer blocks	L	4	8	12
Residual width d	$64L$	256	512	768
Query heads H	L	4	8	12
Key/value heads	L	4	8	12
Head dimension d_h	64	64	64	64
MLP width d_{ff}	$3.5d = 224L$	896	1792	2688

These choices change the following parts of the model together:

- **Block stack:** there are L attention modules, L MLPs, and $2L$ residual additions. Each block has two pre-norm modules and, when enabled, its own query and key normalization modules.
- **Attention:** more heads are added, while each head stays 64-dimensional. All four combined projections W_Q, W_K, W_V, W_O grow with shape $d \times d$.
- **MLPs:** the gate and up matrices have shape $d \times d_{\text{ff}}$, and the down matrix has shape $d_{\text{ff}} \times d$; both dimensions grow.
- **Embeddings and normalization:** the input table grows to $V \times d$ and the unembedding to $d \times V$. Pre-norm and final RMSNorm scale vectors have length d ; each QK norm scale vector remains length 64. There is still just one final RMSNorm.
- **Initialization:** the same rules are reused, but their numerical scales change with width: projection scales depend on $1/\sqrt{d_{\text{in}}}$. With the default untied embeddings, the input table scales as $1/d$ and the output table as $1/\sqrt{d}$. There is no additional depth-dependent residual multiplier.

Vocabulary size (4,096), context length (1,024), the position limit, and RoPE frequencies stay fixed. So do the choices of RMSNorm, SwiGLU, pre-norm, QK normalization, weight tying, and dropout unless explicitly changed. Changing `model_name` does not automatically change learning rate, batch size, data, or training duration in optimizer steps. Larger presets nevertheless require more computation per step. Because depth and width grow together along this ladder, the parameter count grows roughly as d^3 (equivalently, L^3).

For a *depth-only* comparison, supply a custom `model_config` and change only `num_hidden_layers`, keeping width, head counts, and MLP width fixed. Comparing two presets instead changes all the dimensions above.